```
import pandas as pd

df = pd.read_csv('/content/winequality-red.csv', sep=';')

print("Dataset loaded successfully!")
print("Shape:", df.shape)
df.head()
```

```
Dataset loaded successfully!
Shape: (1599, 12)
```

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	alcohol	quality
0	7.4	0.70	0.00	1.9	0.076	11.0	34.0	0.9978	3.51	0.56	9.4	5
1	7.8	0.88	0.00	2.6	0.098	25.0	67.0	0.9968	3.20	0.68	9.8	5
2	7.8	0.76	0.04	2.3	0.092	15.0	54.0	0.9970	3.26	0.65	9.8	5
3	11.2	0.28	0.56	1.9	0.075	17.0	60.0	0.9980	3.16	0.58	9.8	6
4	7.4	0.70	0.00	1.9	0.076	11.0	34.0	0.9978	3.51	0.56	9.4	5

Next steps: Generate code with `df` New interactive sheet

```
# Dataset information
print("Dataset Shape:", df.shape)

print("\nColumn Names:")
print(df.columns.tolist())

print("\nDataset Information:")
df.info()

print("\nFirst 5 Rows:")
display(df.head())
```

```
Dataset Shape: (1599, 12)

Column Names:
['fixed acidity', 'volatile acidity', 'citric acid', 'residual sugar', 'chlorides', 'free sulfur dioxide', 'tot

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1599 entries, 0 to 1598
Data columns (total 12 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   fixed acidity         1599 non-null   float64
 1   volatile acidity      1599 non-null   float64
 2   citric acid           1599 non-null   float64
 3   residual sugar        1599 non-null   float64
 4   chlorides             1599 non-null   float64
 5   free sulfur dioxide   1599 non-null   float64
 6   total sulfur dioxide  1599 non-null   float64
 7   density               1599 non-null   float64
 8   pH                    1599 non-null   float64
 9   sulphates             1599 non-null   float64
 10  alcohol               1599 non-null   float64
 11  quality               1599 non-null   int64
dtypes: float64(11), int64(1)
memory usage: 150.0 KB

First 5 Rows:
```

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	alcohol	quality
0	7.4	0.70	0.00	1.9	0.076	11.0	34.0	0.9978	3.51	0.56	9.4	5
1	7.8	0.88	0.00	2.6	0.098	25.0	67.0	0.9968	3.20	0.68	9.8	5
2	7.8	0.76	0.04	2.3	0.092	15.0	54.0	0.9970	3.26	0.65	9.8	5
3	11.2	0.28	0.56	1.9	0.075	17.0	60.0	0.9980	3.16	0.58	9.8	6
4	7.4	0.70	0.00	1.9	0.076	11.0	34.0	0.9978	3.51	0.56	9.4	5

```
print("Missing values in each column:")
print(df.isnull().sum())

print("\nTotal missing values:", df.isnull().sum().sum())

print("\nDuplicate rows:", df.duplicated().sum())
```

```
Missing values in each column:
fixed acidity           0
volatile acidity        0
citric acid             0
residual sugar          0
chlorides               0
free sulfur dioxide     0
total sulfur dioxide    0
density                 0
pH                      0
sulphates               0
alcohol                 0
quality                 0
dtype: int64

Total missing values: 0

Duplicate rows: 240
```

```
print("Quality Score Distribution:")
print(df["quality"].value_counts().sort_index())

print("\nQuality Score Percentages:")
print((df["quality"].value_counts(normalize=True).sort_index() * 100).round(2))
```

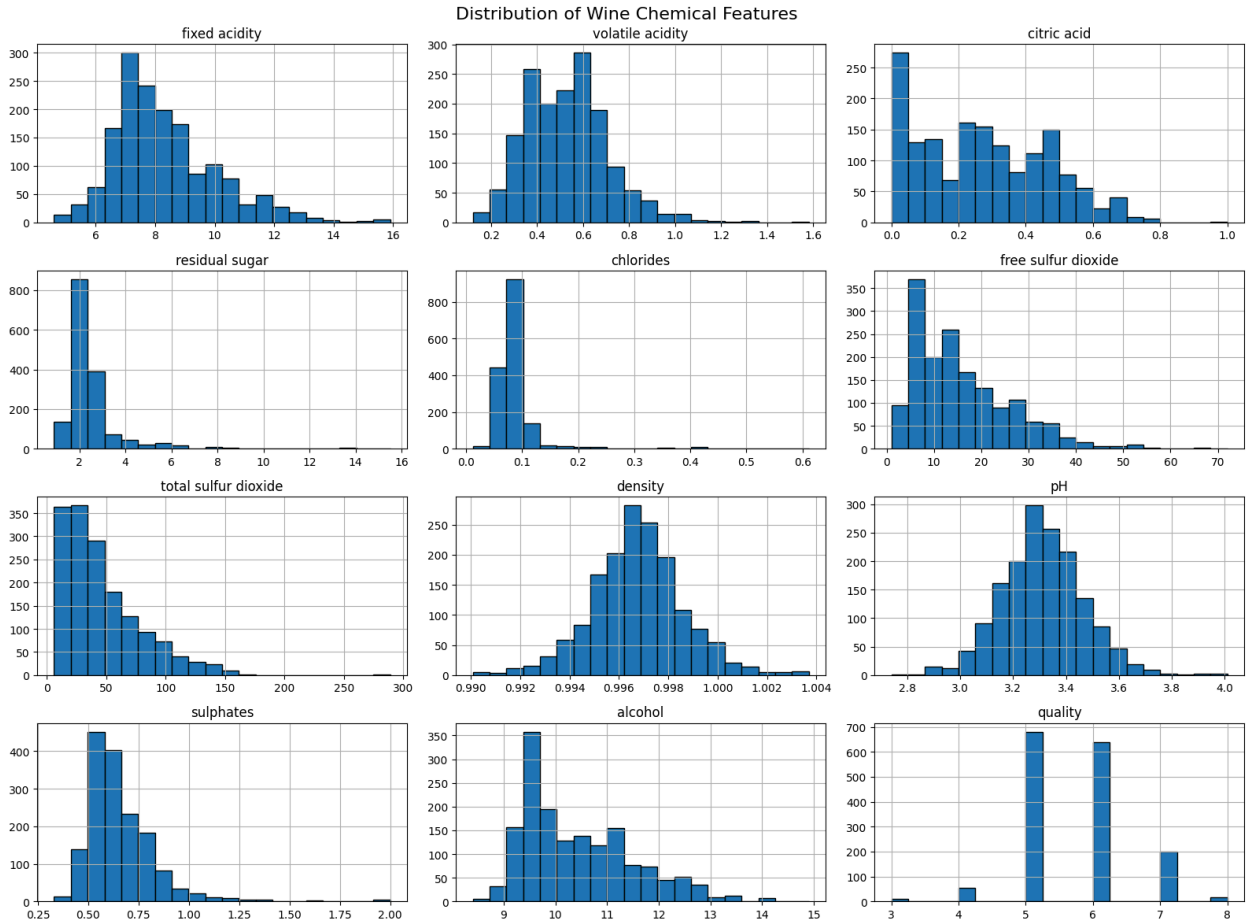
```
Quality Score Distribution:
quality
3     10
4     53
5    681
6    638
7    199
8     18
Name: count, dtype: int64

Quality Score Percentages:
quality
3     0.63
4     3.31
5    42.59
6    39.90
7    12.45
8     1.13
Name: proportion, dtype: float64
```

```
import matplotlib.pyplot as plt

# Distribution plots for all chemical features
df.hist(figsize=(16, 12), bins=20, edgecolor="black")

plt.suptitle("Distribution of Wine Chemical Features", fontsize=16)
plt.tight_layout()
plt.show()
```




```
import seaborn as sns
import matplotlib.pyplot as plt

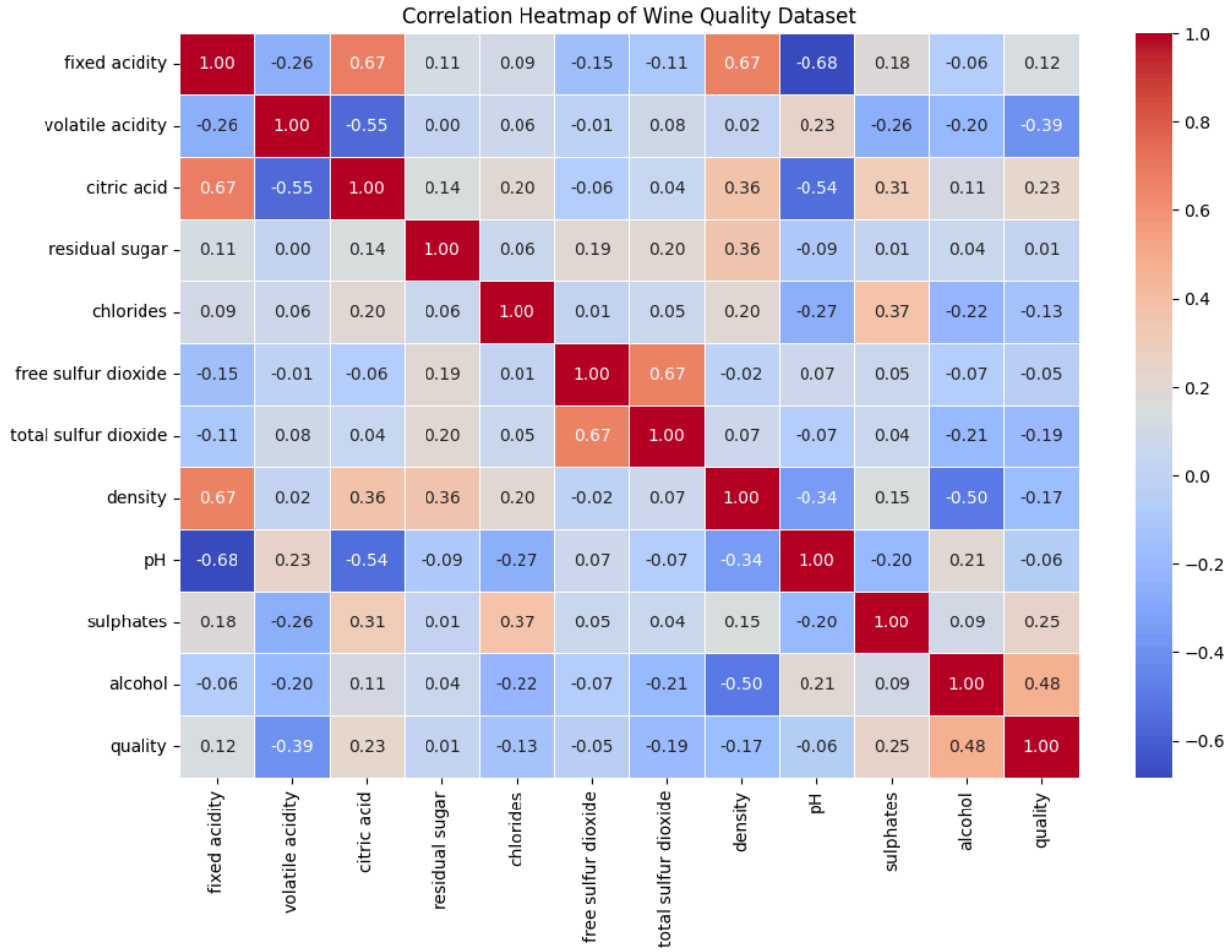
plt.figure(figsize=(12, 8))

correlation = df.corr()

sns.heatmap(
    correlation,
    annot=True,
```

```
    cmap="coolwarm",
    fmt=".2f",
    linewidths=0.5
)

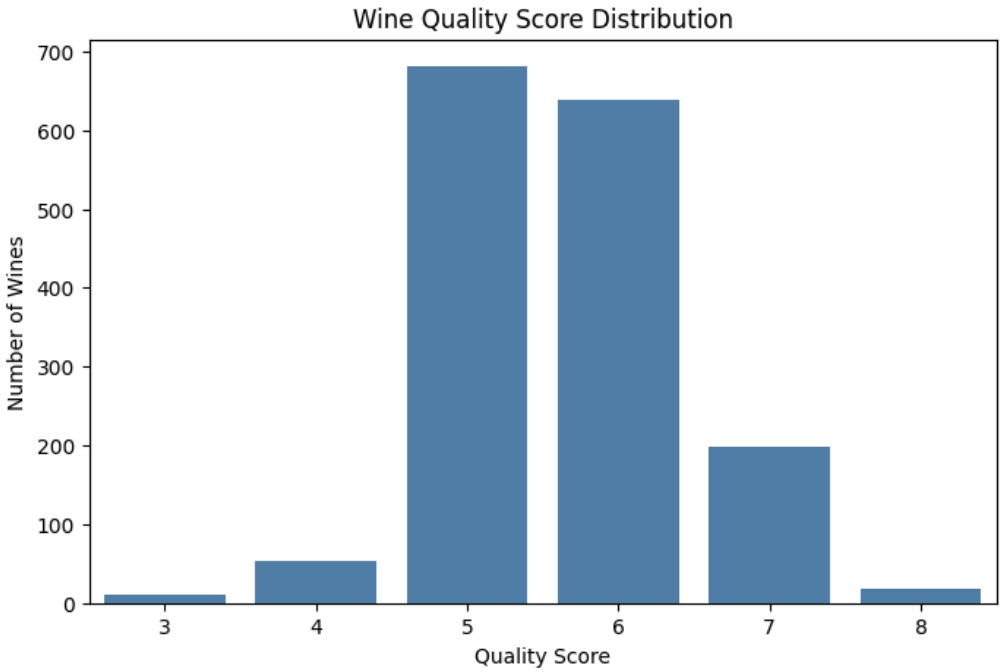
plt.title("Correlation Heatmap of Wine Quality Dataset")
plt.show()
```




```
plt.figure(figsize=(8, 5))

sns.countplot(data=df, x="quality", color="steelblue")

plt.title("Wine Quality Score Distribution")
plt.xlabel("Quality Score")
plt.ylabel("Number of Wines")
plt.show()
```




```
# Convert quality scores into 3 categories

def categorize_quality(quality):
    if quality <= 4:
        return "Low"
    elif quality <= 6:
        return "Medium"
    else:
        return "High"

df["quality_category"] = df["quality"].apply(categorize_quality)

print("Quality categories created successfully!")
print(df[["quality", "quality_category"]].head(10))

print("\nCategory distribution:")
print(df["quality_category"].value_counts())

print("\nCategory percentages:")
print(
    (df["quality_category"].value_counts(normalize=True) * 100).round(2)
)
```

```
Quality categories created successfully!
   quality quality_category
0        5           Medium
1        5           Medium
2        5           Medium
3        6           Medium
4        5           Medium
5        5           Medium
6        5           Medium
7        7             High
8        7             High
9        5           Medium

Category distribution:
quality_category
Medium    1319
High       217
Low         63
Name: count, dtype: int64

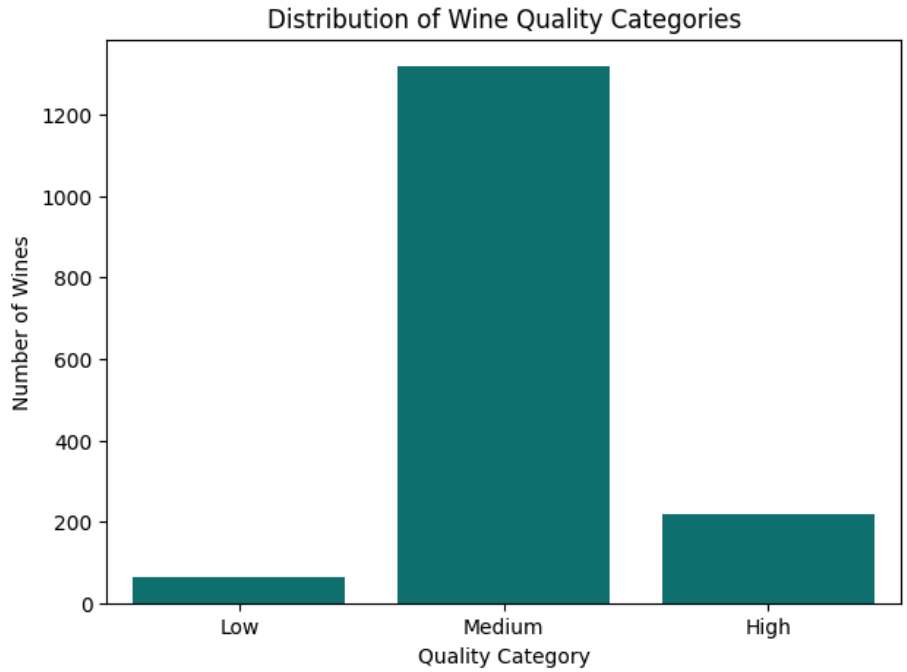
Category percentages:
quality_category
Medium    82.49
High      13.57
```

```
Low        3.94
Name: proportion, dtype: float64
```

```
plt.figure(figsize=(7, 5))

sns.countplot(
    data=df,
    x="quality_category",
    order=["Low", "Medium", "High"],
    color="teal"
)

plt.title("Distribution of Wine Quality Categories")
plt.xlabel("Quality Category")
plt.ylabel("Number of Wines")
plt.show()
```




```
# Chemical properties as input features
X = df.drop(columns=["quality", "quality_category"])

# Target variable
y = df["quality_category"]

print("Features shape:", X.shape)
print("Target shape:", y.shape)
print("\nInput features:")
print(X.columns.tolist())
```

```
Features shape: (1599, 11)
Target shape: (1599,)

Input features:
['fixed acidity', 'volatile acidity', 'citric acid', 'residual sugar', 'chlorides', 'free sulfur dioxide', 'tot
```

```
from sklearn.model_selection import train_test_split

# Split dataset into training and testing sets

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
```

```
)

print("Data split successfully!")

print("\nTraining features:", X_train.shape)
print("Testing features:", X_test.shape)

print("\nTraining target:", y_train.shape)
print("Testing target:", y_test.shape)
```

```
Data split successfully!

Training features: (1279, 11)
Testing features: (320, 11)

Training target: (1279,)
Testing target: (320,)
```

```
print("Training class proportions:")
print((y_train.value_counts(normalize=True) * 100).round(2))

print("\nTesting class proportions:")
print((y_test.value_counts(normalize=True) * 100).round(2))
```

```
Training class proportions:
quality_category
Medium    82.49
High      13.60
Low        3.91
Name: proportion, dtype: float64

Testing class proportions:
quality_category
Medium    82.50
High      13.44
Low        4.06
Name: proportion, dtype: float64
```

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("Feature scaling completed!")
print("Scaled training data:", X_train_scaled.shape)
print("Scaled testing data:", X_test_scaled.shape)
```

```
Feature scaling completed!
Scaled training data: (1279, 11)
Scaled testing data: (320, 11)
```

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, accuracy_score

# Initialize the Random Forest model with 200 estimators and balanced class weights
rf_model = RandomForestClassifier(
    n_estimators=200,
    random_state=42,
    class_weight="balanced"
)

# Fit on scaled training data
rf_model.fit(X_train_scaled, y_train)

# Predict on scaled test data
rf_predictions = rf_model.predict(X_test_scaled)

print("Random Forest model trained successfully!")
print("Predictions:", rf_predictions[:10])

# Print metrics evaluation
print("\nAccuracy Score:", accuracy_score(y_test, rf_predictions))
```

```
print("\nClassification Report:")
print(classification_report(y_test, rf_predictions, zero_division=0))
```

```
Random Forest model trained successfully!
Predictions: ['Medium' 'Medium' 'Medium' 'Medium' 'Medium' 'Medium' 'Medium' 'High'
 'Medium' 'Medium']

Accuracy Score: 0.875

Classification Report:
              precision    recall  f1-score   support

        High       0.75      0.56      0.64        43
         Low       0.00      0.00      0.00        13
      Medium       0.89      0.97      0.93       264

    accuracy                           0.88       320
   macro avg       0.55      0.51      0.52       320
weighted avg       0.83      0.88      0.85       320
```

```
from sklearn.linear_model import SGDClassifier

# Model 2: Stochastic Gradient Descent

sgd_model = SGDClassifier(
    loss="hinge",
    max_iter=2000,
    tol=1e-3,
    random_state=42,
    class_weight="balanced"
)

sgd_model.fit(X_train_scaled, y_train)

sgd_predictions = sgd_model.predict(X_test_scaled)

print("SGD Classifier trained successfully!")
print("Predictions:", sgd_predictions[:10])
```

```
SGD Classifier trained successfully!
Predictions: ['Medium' 'Medium' 'Medium' 'High' 'Medium' 'Medium' 'Low' 'High' 'Medium'
 'Medium']
```

```
from sklearn.metrics import classification_report, accuracy_score

# Evaluate the SGD Classifier
sgd_accuracy = accuracy_score(y_test, sgd_predictions)

print("SGD Classifier Evaluation")
print(f"Accuracy Score: {sgd_accuracy:.4f}")
print("\nClassification Report:")
print(classification_report(y_test, sgd_predictions, zero_division=0))
```

```
SGD Classifier Evaluation
Accuracy Score: 0.7781

Classification Report:
              precision    recall  f1-score   support

        High       0.49      0.42      0.45        43
         Low       0.09      0.15      0.11        13
      Medium       0.88      0.87      0.87       264

    accuracy                           0.78       320
   macro avg       0.48      0.48      0.48       320
weighted avg       0.79      0.78      0.78       320
```

```
from sklearn.svm import SVC

# Model 3: Support Vector Classifier

svc_model = SVC(
    kernel="rbf",
    class_weight="balanced"
)
```

```
svc_model.fit(X_train_scaled, y_train)

svc_predictions = svc_model.predict(X_test_scaled)

print("SVC model trained successfully!")
print("Predictions:", svc_predictions[:10])
```

```
SVC model trained successfully!
Predictions: ['Low' 'Medium' 'Medium' 'High' 'Medium' 'Medium' 'Medium' 'High' 'Medium'
 'Medium']
```

```
from sklearn.metrics import classification_report, accuracy_score

# Evaluate the SVC model
svc_accuracy = accuracy_score(y_test, svc_predictions)

print("Support Vector Classifier (SVC) Evaluation")
print(f"Accuracy Score: {svc_accuracy:.4f}")
print("\nClassification Report:")
print(classification_report(y_test, svc_predictions, zero_division=0))
```

```
Support Vector Classifier (SVC) Evaluation
Accuracy Score: 0.7094

Classification Report:
              precision    recall  f1-score   support

        High       0.40      0.74      0.52        43
         Low       0.10      0.23      0.14        13
      Medium       0.92      0.73      0.81       264

    accuracy                           0.71       320
   macro avg       0.47      0.57      0.49       320
weighted avg       0.82      0.71      0.74       320
```

```
print("Random Forest:", rf_model.score(X_test, y_test))
print("SGD Classifier:", sgd_model.score(X_test_scaled, y_test))
print("SVC:", svc_model.score(X_test_scaled, y_test))
```

```
Random Forest: 0.825
SGD Classifier: 0.778125
SVC: 0.709375
/usr/local/lib/python3.13/dist-packages/sklearn/utils/validation.py:2732: UserWarning: X has feature names, but
  warnings.warn(
```

```
from sklearn.metrics import accuracy_score, classification_report

models_predictions = {
    "Random Forest": rf_predictions,
    "SGD Classifier": sgd_predictions,
    "SVC": svc_predictions
}

for name, predictions in models_predictions.items():
    print("=" * 60)
    print(name)
    print("=" * 60)

    print("Accuracy:", round(accuracy_score(y_test, predictions), 4))
    print("\nClassification Report:")
    print(classification_report(y_test, predictions, zero_division=0))
```

```
============================================================
Random Forest
============================================================
Accuracy: 0.875

Classification Report:
              precision    recall  f1-score   support
```

```
              precision    recall  f1-score   support

        High       0.75      0.56      0.64        43
         Low       0.00      0.00      0.00        13
      Medium       0.89      0.97      0.93       264

    accuracy                           0.88       320
   macro avg       0.55      0.51      0.52       320
weighted avg       0.83      0.88      0.85       320

============================================================
SGD Classifier
============================================================
Accuracy: 0.7781

Classification Report:
              precision    recall  f1-score   support

        High       0.49      0.42      0.45        43
         Low       0.09      0.15      0.11        13
      Medium       0.88      0.87      0.87       264

    accuracy                           0.78       320
   macro avg       0.48      0.48      0.48       320
weighted avg       0.79      0.78      0.78       320

============================================================
SVC
============================================================
Accuracy: 0.7094

Classification Report:
              precision    recall  f1-score   support

        High       0.40      0.74      0.52        43
         Low       0.10      0.23      0.14        13
      Medium       0.92      0.73      0.81       264

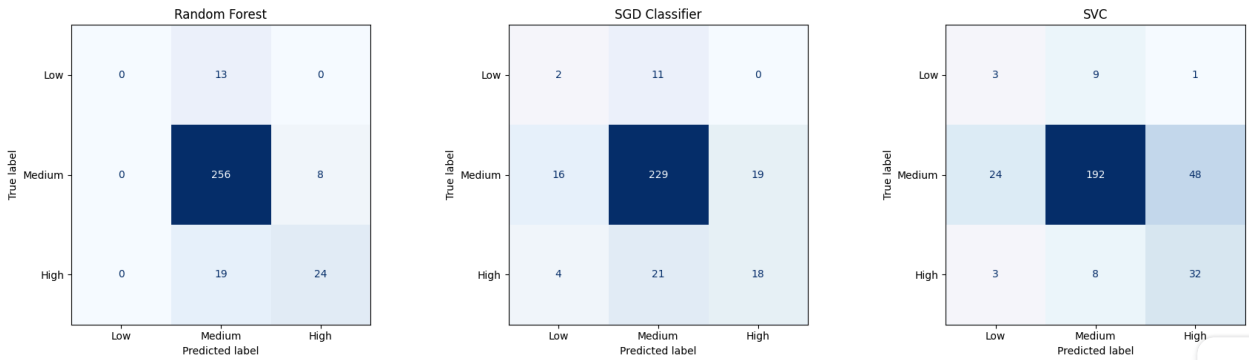
    accuracy                           0.71       320
   macro avg       0.47      0.57      0.49       320
weighted avg       0.82      0.71      0.74       320
```

```
from sklearn.metrics import ConfusionMatrixDisplay

fig, axes = plt.subplots(1, 3, figsize=(18, 5))

for ax, (name, predictions) in zip(axes, models_predictions.items()):
    ConfusionMatrixDisplay.from_predictions(
        y_test,
        predictions,
        labels=["Low", "Medium", "High"],
        display_labels=["Low", "Medium", "High"],
        cmap="Blues",
        ax=ax,
        colorbar=False
    )
    ax.set_title(name)

plt.tight_layout()
plt.show()
```



```
from sklearn.metrics import f1_score, precision_score, recall_score

results = []

for name, predictions in models_predictions.items():
    results.append({
        "Model": name,
        "Accuracy": accuracy_score(y_test, predictions),
        "Weighted Precision": precision_score(
            y_test, predictions, average="weighted", zero_division=0
        ),
        "Weighted Recall": recall_score(
            y_test, predictions, average="weighted", zero_division=0
        ),
        "Weighted F1-Score": f1_score(
            y_test, predictions, average="weighted", zero_division=0
        ),
        "Macro F1-Score": f1_score(
            y_test, predictions, average="macro", zero_division=0
        )
    })

comparison_df = pd.DataFrame(results).sort_values(
    by="Weighted F1-Score", ascending=False
)

display(comparison_df.round(4))
```

	Model	Accuracy	Weighted Precision	Weighted Recall	Weighted F1-Score	Macro F1-Score
0	Random Forest	0.8750	0.8341	0.8750	0.8512	0.5225
1	SGD Classifier	0.7781	0.7929	0.7781	0.7848	0.4789
2	SVC	0.7094	0.8150	0.7094	0.7448	0.4892

```
# Get feature importance from Random Forest

importance_df = pd.DataFrame({
    "Feature": X.columns,
    "Importance": rf_model.feature_importances_
})

importance_df = importance_df.sort_values(
    by="Importance",
    ascending=False
)

print("Feature Importance Ranking:")
display(importance_df.round(4))

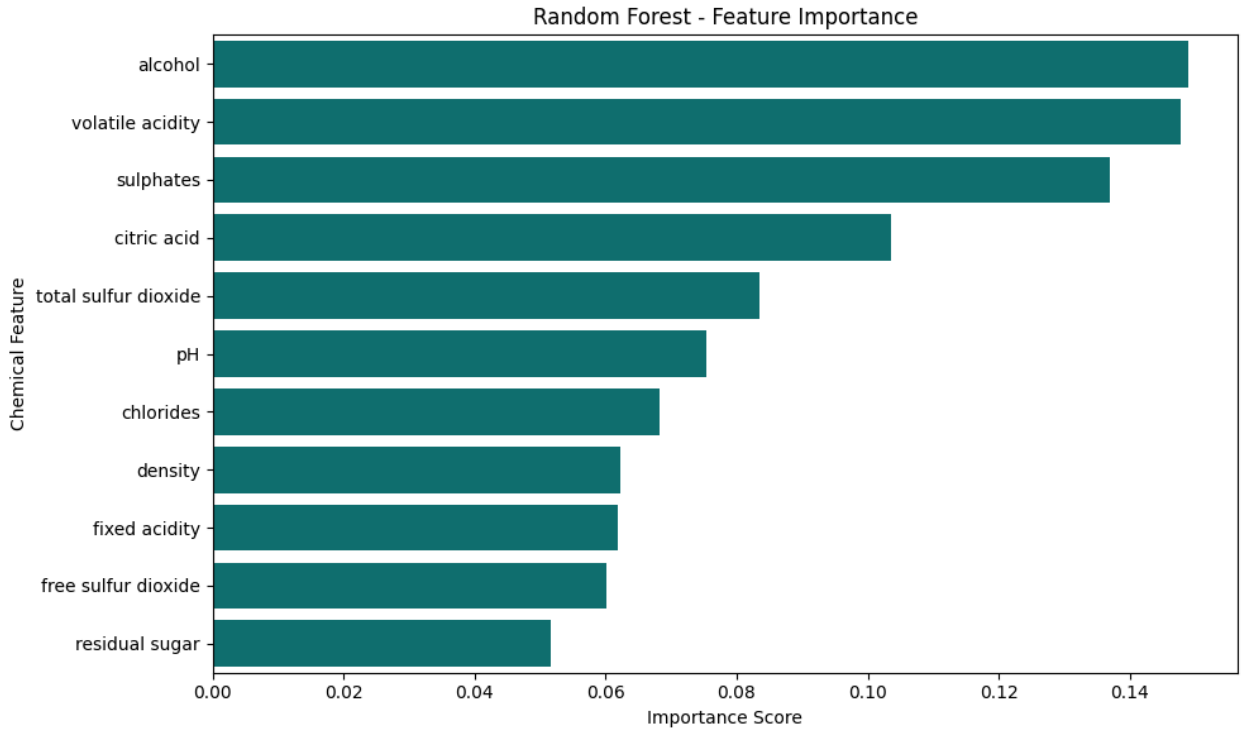
# Plot feature importance
plt.figure(figsize=(10, 6))

sns.barplot(
    data=importance_df,
    x="Importance",
    y="Feature",
    color="teal"
)

plt.title("Random Forest - Feature Importance")
plt.xlabel("Importance Score")
plt.ylabel("Chemical Feature")
plt.tight_layout()
plt.show()
```

```
Feature Importance Ranking:
```

	Feature	Importance
10	alcohol	0.1490
1	volatile acidity	0.1478
9	sulphates	0.1369
2	citric acid	0.1035
6	total sulfur dioxide	0.0834
8	pH	0.0753
4	chlorides	0.0682
7	density	0.0622
0	fixed acidity	0.0618
5	free sulfur dioxide	0.0601
3	residual sugar	0.0516




```
print("Top 5 Important Features:")

for i, row in enumerate(importance_df.head(5).itertuples(index=False), start=1):
    print(f"{i}. {row.Feature}: {row.Importance:.4f}")
```

```
Top 5 Important Features:
1. alcohol: 0.1490
2. volatile acidity: 0.1478
3. sulphates: 0.1369
4. citric acid: 0.1035
5. total sulfur dioxide: 0.0834
```

```
# Compare model performance

comparison_df.set_index("Model")[
    ["Accuracy", "Weighted F1-Score", "Macro F1-Score"]
].plot(kind="bar", figsize=(10, 6), ylim=(0, 1))

plt.title("Performance Comparison of Classification Models")
plt.ylabel("Score")
plt.xlabel("Model")
```

```
plt.xticks(rotation=0)
plt.legend(loc="lower right")
plt.tight_layout()
plt.show()
```

